

NamingContest.com — Going Live & Environment Guide

Day-one setup, every configuration value, fresh environments, and the launch checklist

The procedures document. The Developer Handbook explains how the system works; this one is what you actually type — day-one local setup, where every configuration value comes from, building a fresh environment from zero, and the launch-day checklist.

No credentials appear in this document. Every value is fetched from the relevant dashboard by whoever holds access.

1. Day one: run it locally

Prerequisites: Node 18+ (built on 24), npm, git.

```
git clone <repo-url>
cd COMPLETE-FINAL
npm install
cp .env.example .env      # fill in — see the table in §3
npm run dev               # Vite on http://localhost:5173
```

Leave `VITE_BETA_PASSWORD` unset locally: unset means the beta gate is off.

Before any deploy: `npm run build` must pass. There is no CI — the build is the only gate, run it every time.

Branch and deploy workflow

- `backend` — working branch. Everything lands here first.
- `master` — production. **Pushing master auto-deploys to Vercel.**
- Promote with a no-fast-forward merge:

```
git checkout master && git merge --no-ff backend && git push
git checkout backend
```

- Any branch push gets a Vercel preview deployment (this is staging).

- Verifying a deploy landed: the SPA rewrite returns HTTP 200 + index.html for *every* path, so status codes prove nothing. Fetch the homepage, take the hashed `/assets/index-*.js` filename, and grep it for a string from your change.

Definition of "it works" — the smoke test

Run after any significant change, and after any environment work:

1. Homepage loads; beta gate accepts the password (if set)
2. Create a contest end to end, pay with `4242 4242 4242 4242`
3. Exactly **one** "Your contest is live" email arrives; for a guest launch its button signs you in and lands on the contest
4. Stripe → Payments shows the charge; Webhooks shows a delivered `payment_intent.succeeded` with HTTP 200
5. Open the invite link in a private window, join with a second email, submit a name
6. Fast-forward to voting (see `TESTING-EMAILS.md`), vote, close, crown — each transition sends its email

2. The six things that hold configuration

WHERE	WHAT LIVES THERE
Vercel env vars	The four <code>VITE_*</code> values — build-time inlined, changing one requires a redeploy
Supabase function secrets	Runtime server secrets — take effect immediately, no redeploy
Supabase Vault	<code>notify_secret</code> (the DB trigger's copy of NOTIFY_SECRET)
Supabase Auth settings	SMTP, redirect allow-list, the two email templates
Stripe dashboard	API keys and webhook endpoints — separate per test/live mode
eNom DNS	Domain records; email records for Resend live on <code>send.</code> and <code>resend._domainkey</code>

3. Where every value comes from

VARIABLE	LIVES IN	FETCH IT FROM
VITE_SUPABASE_URL	Vercel + .env	Supabase → Project Settings → API → Project URL
VITE_SUPABASE_ANON_KEY	Vercel + .env	Same page → anon public key
VITE_STRIPE_PUBLISHABLE_KEY	Vercel + .env	Stripe → Developers → API keys (pk_test_ / pk_live_ per mode)
VITE_BETA_PASSWORD	Vercel	Chosen string; unset = gate off (it ships in the JS bundle — obscurity, not security)
STRIPE_SECRET_KEY	Supabase secrets	Stripe → Developers → API keys (sk_test_ / sk_live_) — dashboard to dashboard, never through a chat or document
STRIPE_WEBHOOK_SECRET	Supabase secrets	Stripe → Developers → Webhooks → the endpoint → Signing secret (whsec_ , per endpoint, per mode)
RESEND_API_KEY	Supabase secrets + Auth SMTP password	Resend → API Keys
NOTIFY_SECRET	Supabase secrets and Vault	Self-generated random string — both copies must match or notify returns 401
SITE_URL	Supabase secrets	https://namingcontest.com (base for links in emails)
CONTACT_TO	Supabase secrets	Inbox that receives contact-form messages
SUPABASE_URL , SUPABASE_SERVICE_ROLE_KEY	—	Injected into edge functions automatically; never set by hand, never in the client

Setting a function secret:

```
npx supabase secrets set NAME=value --project-ref kgcggyuoeyaygyawnlcs
```

or Supabase dashboard → Project Settings → Edge Functions → Secrets.

4. Going live — the launch checklist

Do these **in order**. Steps 3–5 are the ones that cost money if skipped.

1. Preconditions (client-side, from the Platform Guide) - hello@namingcontest.com mailbox receives mail (eNom forwarding tested) - Landing testimonials are real; social handles registered or icons removed - Senior developer review complete

2. Stripe account is activated Live keys only work on an activated account: business details, identity, and a **bank account for payouts** (Stripe → Settings → Business). Do this days ahead — activation can involve a review delay.

3. Switch the Stripe keys - Stripe → toggle **Test mode OFF** → Developers → API keys - `sk_live_...` → Supabase secret `STRIPE_SECRET_KEY` (runtime, immediate) - `pk_live_...` → Vercel env `VITE_STRIPE_PUBLISHABLE_KEY` → **redeploy** (build-time inlined; without the redeploy the site still uses the test key)

4. ⚠ Create the LIVE webhook endpoint Webhook endpoints are **per mode**. The existing endpoint lives in test mode and will never see a live payment. In **live mode**: - Developers → Webhooks → Add endpoint - URL: `https://kgcggyuoeyaygyawnlcs.supabase.co/functions/v1/stripe-webhook` - Event: `payment_intent.succeeded` only - Copy the new endpoint's `whsec_` → Supabase secret `STRIPE_WEBHOOK_SECRET`

Miss this step and cards still charge and contests still launch through the browser path — but the backstop is silently gone, and a buyer whose tab closes mid-payment is charged with nothing delivered.

5. Verify with one real payment Launch a \$9 contest with a real card. Confirm: contest goes live, exactly one receipt email, live-mode Payments shows the charge, live-mode Webhooks shows a 200 delivery. Then refund yourself: Payments → the charge → Refund.

6. Open the doors - Delete `VITE_BETA_PASSWORD` from Vercel env → redeploy. Unset = gate off.

7. Same-day aftercare - Watch Stripe Payments and the webhook log for the first real customers - Watch Resend for bounces - DMARC is `p=none` (monitoring only). After a few weeks of clean reports, tighten the `_dmarc` TXT record to `p=quarantine` for deliverability

Rollback: switching the two Stripe keys back to `sk_test_` / `pk_test_` (+ redeploy for the publishable key) restores test mode. Nothing else needs touching.

5. Building a fresh environment from zero

For a true staging clone or a rebuild. Order matters.

1. Supabase project — create one, note its project ref.

2. Extensions — the migrations run `create extension if not exists` for `pg_cron` and `pg_net`; if a migration fails on one, enable it first under Database → Extensions.

3. Migrations — SQL editor, run `supabase/migrations/0001...0023` **in order**, each file whole. They also create the storage bucket (0002) and schedule both cron jobs (0009, 0020).

4. ⚠ The one hardcoded value — the notify trigger (`notify_contest_change`, latest version in 0023) contains the production project ref in its URL:

```
v_url := 'https://kgcggyuoeyaygyawnlcs.supabase.co/functions/v1/notify'
```

On a fresh project, edit that line to the new ref before running, or lifecycle emails will be delivered to production's notify function.

5. Vault — `select vault.create_secret('<random string>', 'notify_secret');` and set the same string as the `NOTIFY_SECRET` function secret.

6. Edge functions — deploy all six; the JWT flag is part of the design:

```
npx supabase functions deploy launch-contest --no-verify-jwt --project-ref <REF>
npx supabase functions deploy notify --no-verify-jwt --project-ref <REF>
npx supabase functions deploy contact --no-verify-jwt --project-ref <REF>
npx supabase functions deploy stripe-webhook --no-verify-jwt --project-ref <REF>
npx supabase functions deploy create-payment-intent --project-ref <REF>
npx supabase functions deploy confirm-launch --project-ref <REF>
```

(The four `--no-verify-jwt` ones authenticate their own way: rate limits + validation, the notify secret, or the Stripe signature.)

7. Function secrets — all six from §3.

8. Auth configuration (Supabase → Authentication): - **SMTP**: host `smtp.resend.com`, port `465`, username `resend`, password = the Resend API key, sender `noreply@namingcontest.com`, sender name `NamingContest` - **URL Configuration**: Site URL = the environment's domain; Redirect URLs must include `<domain>/**` and `http://localhost:5173/**`. Without the wildcard, magic links to specific pages (e.g. a contest URL) fail silently and fall back - **Email templates**: paste `supabase/templates/magic-link.html` and `confirm-signup.html` into their slots (source view, replace-all). Subjects: `Your sign-in link - NamingContest` / `Welcome to NamingContest - your sign-in link`

9. Vercel — import the repo, set the four `VITE_*` vars, deploy, attach the domain.

10. Stripe — webhook endpoint for this environment (§4 step 4, test mode for staging).

11. Run the smoke test from §1. A fresh environment isn't done until it passes end to end.

6. Routine operations quick reference

TASK	HOW
Deploy app change	merge <code>backend</code> → <code>master</code> , push
Deploy function change	<code>npx supabase functions deploy <name> [--no-verify-jwt] --project-ref kgcggyuoeyaygyawnlcs</code>
Change a server secret	Supabase secrets (immediate)
Change a <code>VITE_*</code> value	Vercel env + redeploy
Schema change	New migration file <code>NNNN_slug.sql</code> , run in SQL editor, commit the file
Fast-forward a test contest	<code>TESTING-EMAILS.md</code>
Check cron	<code>select jobname, schedule, active from cron.job;</code>
Check trigger→function calls	<code>select * from net._http_response order by id desc limit 5;</code>
Function logs	Supabase → Edge Functions → the function → Logs